

Git & GitHub for Data Science

Target outcome:

Participants stop zipping files and start working like professionals.



Why Version Control Matters

Key messages

- Git is a time machine
- Git is a collaboration contract
- Git protects you from yourself
- **The "Final_v2_REALLY_FINAL.ipynb" Problem:** Avoid the mess of multiple file versions.
- **Experimentation:** Test new features or hyperparameters without breaking your working model.
- **Collaboration:** Multiple scientists can work on the same codebase simultaneously.
- **Accountability:** Track exactly who changed which line of code and why.

Git Basics (The Local Workflow)

- ***git init***: Initializes a new Git repository in your folder.
- ***git status***: Your "What's going on?" command. Shows untracked/modified files.
- ***git add <file>***: Moves changes to the **Staging Area** (the "loading dock").
- ***git commit -m "message"***: Records a snapshot of your staged changes.
- ***git log***: Shows the history of your commits.

GitHub & The Cloud

- **Remote vs. Local:** Your PC has the files; GitHub hosts the backup and collaboration hub.
- Creating repositories
- **The "Link":** Connect your local repo to GitHub using `git remote add origin <url>`.
- ***git push*:** Sends your local commits to the cloud.
- ***git pull*:** Grabs the latest changes from the cloud to your local machine.

Branching & Merging

- **The "Main" Branch:** Should always be your stable, production-ready code.
- **Feature Branches:** Create a branch for a new experiment (e.g., *git checkout -b xgboost-tuning*).
- **Merging:** Once the experiment works, "merge" it back into the Main branch.

Documentation & Best Practices

- **Effective Commits:**
 - Use the imperative mood ("Add preprocessing script" instead of "I added some stuff").
 - Bad habit: Edit five things. Commit once
 - Good habit: Make one meaningful change. Commit it
- **README.md:** This is your project's storefront. It must include:
 - Installation instructions.
 - Data sources.
 - How to run the analysis/model.

Hands-On Lab: Putting it into Practice



Step 1: Setup & Initial Commit

- Open your terminal/command prompt in your project folder.
- `git init`
- Create a file named `.gitignore` and add `data/` or `*.csv` (you don't want to upload huge datasets!).
- `git add .`
- `git commit -m "Initial commit: Week 1-2 EDA and preprocessing scripts"`

Step 2: Connect to GitHub & Step 3: The README Challenge

- **Connect to GitHub**
 - Go to GitHub and create a new repository (don't initialize with a README).
 - Copy the URL and run: `git remote add origin <YOUR_URL>` `git push -u origin main`
- **Create a README.md file in your folder. Use this structure:**
 - Project Title
 - Overview: Briefly describe the ML problem you're solving.
 - Key Findings
 - Insight 1
 - Insight 2
 - How to Run: ``python main.py``
- `git add README.md -> git commit -m "Add project documentation" -> git push`

Step 4: Collaborative Branching (Pairs)

- **Person A:** Invite **Person B** as a collaborator in GitHub settings.
- **Person B:** `git clone <URL>`, then create a branch: `git checkout -b feature-improvement`.
- **Person B:** Edit a file, commit, and `git push origin feature-improvement`.
- **Person A:** Go to GitHub, view the "Pull Request," and merge it into the main branch.

